
Aplicación para la navegación en colecciones de
contenidos multimedia
Application for Browsing Multimedia Content
Collections



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Juan Andrés Hibjan Cardona
Leonardo Prado de Souza

Director

José Luis Sierra Rodríguez

Colaborador

Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid

Aplicación para la navegación en
colecciones de contenidos multimedia
Application for Browsing Multimedia
Content Collections

Trabajo de Fin de Grado en Ingeniería Informática

Autor

Juan Andrés Hibjan Cardona
Leonardo Prado de Souza

Director

José Luis Sierra Rodríguez

Colaborador

Convocatoria: *Junio 2026*

Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid

10 de Junio de 2026

Dedicatoria

...

Agradecimientos

...

Resumen

Aplicación para la navegación en colecciones de contenidos multimedia

Un resumen en castellano de media página, incluyendo el título en castellano. A continuación, se escribirá una lista de no más de 10 palabras clave.

Palabras clave

Máximo 10 palabras clave separadas por comas

Abstract

Application for Browsing Multimedia Content Collections

An abstract in English, half a page long, including the title in English. Below, a list with no more than 10 keywords.

Keywords

10 keywords max., separated by commas.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	2
1.3.1. Fase 1: Investigación y Prototipo en Java (Septiembre 2025 - Enero 2026)	2
1.3.2. Fase 2: Desarrollo Web e Integración (Enero 2026 - Marzo 2026)	3
1.3.3. Cronograma de Ejecución y Diagrama Gantt	3
2. Estado de la Cuestión	5
3. Arquitectura y Metodología	7
3.1. Metodología de Ingeniería del Software	7
3.2. Arquitectura del Sistema	9
3.3. Persistencia y Acceso a Datos	10
3.4. Integración API	10
4. Desarrollo e Implementación	11
4.1. Representación del contexto de exploración	11
4.2. Filtrado clásico y relacional	11
4.3. Navegación transversal	11
4.4. Operaciones de conjuntos	11
4.5. Obstáculos encontrados y soluciones implementadas	11
5. Pruebas y Validación	13
5.1. Casos de uso validados	13
6. Conclusiones y Trabajo Futuro	15
Introduction	17
Conclusions and Future Work	19

Contribuciones Personales	21
Bibliografía	23
A. Título del Apéndice A	25
B. Título del Apéndice B	27

Índice de figuras

1.1. Diagrama Gantt TFG	4
-----------------------------------	---

Índice de tablas

Introducción

“Sólo hay una victoria decisiva: la última”
— Carl Von Clausewitz

...

1.1. Motivación

En los últimos tiempos hemos pasado por un gran cambio en el paradigma de consumo digital. El acúmulo cuantitativo de contenidos digitales ha sobrepasado su umbral cualitativo y hoy supone una manera distinta de relacionarse con ello. La abundancia de información genera una "sobrecarga de información" (o *information overload*), en la cual todo y cualquier usuario necesita de una cantidad cada vez mayor de tiempo para navegar. Eso se hace especialmente evidente en las plataformas digitales, en particular las de streaming multimedia (como Netflix, Disney+ y HBO Max), en las que difícilmente encontramos qué consumir sin invertir un tiempo sustancial en ello.

Ese cambio de paradigma, y el subsecuente problema, sigue siendo abordado con métodos tradicionales como: la búsqueda directa (por palabras clave), el filtrado clásico por género o medio y los sistemas de recomendación basados en algoritmos ocultos al consumidor. Estos abordajes son limitados y sirven poco al nuevo contexto de consumo digital. La búsqueda directa es determinista y no permite ninguna flexibilidad, especialmente al suponer que el usuario sabe exactamente lo que está buscando de antemano. El filtrado tradicional suele ser incompleto e inconsistente, usando taxonomías ocultas e inescrutables. Los algoritmos de recomendación suelen servir más como un propagador de la burbuja de filtros (o *filter bubble*), al depender directamente del historial de consumo, o de lo que la plataforma desee promover, quitando la agencia del usuario en última instancia.

Este problema, entretanto, no es exclusivo de experiencia de usuario, sino también supone un problema tecnológico relacionado con la forma en la que se diseñan y utilizan las colecciones de información. Los modelos relacionales de navegación y los sistemas de navegación sobre colecciones (*collection navigation systems*), traen un enfoque distinto en el que los datos no se representan como elementos aislados, sino como entidades conectadas mediante relaciones semánticas. Ese abordaje permite

recoger colecciones de manera interactiva, con mecanimos de navegación más expresivos en los que el usuario puede explorar progresivamente una colección compleja combinando filtros, relaciones y transiciones entre entidades.

Existe, por lo tanto, una necesidad declarada de que los sistemas de navegación acompañen ese cambio de paradigma. Eso solo se puede cumplir permitiendo a los usuarios herramientas de exploración interactiva más ricas y flexibles. Dichas herramientas deben combinar la precisión establecida por la búsqueda directa, pero con una mayor expresividad que el filtrado clásico sin quitar la agencia del usuario.

1.2. Objetivos

Descripción de los objetivos del trabajo.

1.3. Plan de trabajo

Hemos optado por una metodología ágil, basada en su mayor parte en el marco de trabajo **Scrum**, adaptada a un proyecto en pareja. La adopción de ese método viene dada por la naturaleza exploratoria e innovadora del proyecto, ya que los requisitos y el motor de navegación necesitaban un refinamiento y optimización iterativos.

El desarrollo fue organizado mediante *sprints* (ciclos de trabajo entre 2 y 3 semanas), definiendo un backlog inicial (*Product Backlog*) con los requisitos fundamentales del sistema. Además, se realizarían reuniones periódicas de seguimiento (*Sprint Reviews*), que permiten evaluar cada incremento del software, adaptar la planificación de acuerdo con los problemas y obstáculos encontrados, en conjunto con la tutoría del profesor.

Con respecto a la macroplanificación, el proyecto se estructuró en dos grandes fases secuenciales de desarrollo:

1.3.1. Fase 1: Investigación y Prototipo en Java (Septiembre 2025 - Enero 2026)

Con el objetivo de validar completa y rigurosamente la viabilidad del motor de navegación y los algoritmos involucrados, se optó por desarrollar un prototipo en Java. Deliberadamente abandonando la implementación inicial de una interfaz gráfica, hemos centralizado los esfuerzos en su eficacia, eficiencia y robustez técnica de los componentes del sistema. Este enfoque permitió analizar en profundidad el comportamiento del algoritmo planteado y la correcta integración con la base de datos TMDB.

- **Estudio precedente y proceso de diseño arquitectónico:** Es ese momento hemos definido es estado de la cuestión basándonos en artículos de investigación previos, seleccionamos los datasets pertinentes y de dominio público y diseñamos el modelo básico de datos relacional.

- **Desarrollo del motor de navegación (núcleo backend):** Siguiendo con la implementación de la lógica en Java, desarrollamos una máquina de estados (*State*, *StateManager*) con capacidad para: gestionar el historial, los filtros de metadatos, los filtros relacionales y los saltos transversales *Link*.

- **Integración de TMBD:**

Para empezar la validación del motor de navegación, hemos optado por integrar la base de datos pública TMBD (*The Movie Database*), que permite obtener información sobre películas, series y personas. Además de consumir la API, se diseñó un proceso de transformación y almacenamiento de los datos en nuestro modelo relacional, permitiendo adaptar la estructura de TMDB a las necesidades de navegación del motor.

1.3.2. Fase 2: Desarrollo Web e Integración (Enero 2026 - Marzo 2026)

Con el núcleo del modelo de navegación funcional, hemos trasladado el esfuerzo en exponer sus funcionalidades a través de una arquitectura cliente-servidor, además de añadir una interfaz de usuario.

- **Exposición de API RESTful:** Hemos desarrollado una capa de controladores usando Servlets de Java (*API RESTful*) para exponer, con seguridad y de manera estructurada, las capacidades del motor (*FacetsServlet*, *NavigationServlet*, *UnionServlet*, etc.) que gestiona el contexto mediante sesiones de usuario.
- **Desarrollo del Frontend:**

Trás definir la API, se desarrolló una interfaz web para la interacción con el motor de navegación. Dentre sus principales componentes están: el área de resultados, sistema de filtrado dinámico de metadatos, controles de navegación de estados, conjunto de controles de filtros y exploración de relaciones. Todo desde el paradigma de API RESTful con actualización dinámica.
- **Pruebas de Integración y Refinamiento:** Unimos todos los componentes, hemos hecho pruebas de carga con los *datasets* completos y evaluado la experiencia de usuario (*UX*).

1.3.3. Cronograma de Ejecución y Diagrama Gantt

- **Septiembre 2025:** Definición del proyecto, revisión de la literatura (estado del arte) y selección de datasets.
- **Octubre 2025:** Diseño del modelo de datos y desarrollo inicial del motor de navegación (Fase 1).
- **Noviembre 2025:** Implementación completa de la máquina de estados e historial de navegación.

- **Diciembre 2025:** Prototipado de la integración del dataset TMDB y cierre de la Fase 1.
- **Enero 2026:** Diseño e implementación de la API RESTful (controladores y enrutamiento) (Fase 2).
- **Febrero 2026:** Desarrollo del cliente web (Frontend) e integración con el backend.
- **Marzo 2026:** Pruebas de integración, resolución de bugs (obstáculos técnicos) y validación de casos de uso.
- **Abril 2026:** Redacción intensiva de la memoria del Trabajo de Fin de Grado.
- **Mayo 2026:** Revisiones finales de la memoria, preparación de la presentación, ensayo y defensa ante el tribunal universitario.

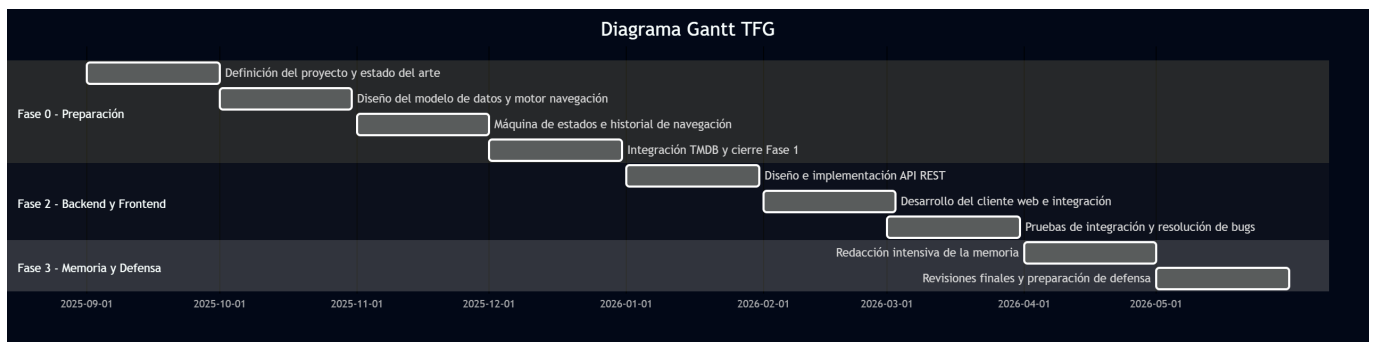


Figura 1.1: Diagrama Gantt TFG

Estado de la Cuestión

En el estado de la cuestión es donde aparecen gran parte de las referencias bibliográficas del trabajo. Una de las formas más cómodas de gestionar la bibliografía en $\text{\LaTeX}$ es utilizando **bibtex**. Las entradas bibliográficas deben estar en un fichero con extensión *.bib* (con esta plantilla se proporciona el fichero *biblio.bib*, donde están las entradas referenciadas más abajo). Cada entrada bibliográfica tiene una clave que permite referenciarla desde cualquier parte del texto con los siguiente comandos:

- Referencia bibliografica con cite: Bautista et al. (1998)
- Referencia bibliográfica con citep: (Oetiker et al., 1996)
- Referencia bibliográfica con citet: Krishnan (2003)

Es posible citar más de una fuente, como por ejemplo (Mittelbach et al., 2004; Lamport, 1994; Knuth, 1986)

Después, $\text{\LaTeX}$ se ocupa de rellenar la sección de bibliografía con las entradas **que hayan sido citadas** (es decir, no con todas las entradas que hay en el *.bib*, sino sólo con aquellas que se hayan citado en alguna parte del texto).

Bibtex es un programa separado de latex, pdflatex o cualquier otra cosa que se use para compilar los *.tex*, de manera que para que se rellene correctamente la sección de bibliografía es necesario compilar primero el trabajo (a veces es necesario compilarlo dos veces), compilar después con bibtex, y volver a compilar otra vez el trabajo (de nuevo, puede ser necesario compilarlo dos veces).

Capítulo 3

Arquitectura y Metodología

Este capítulo centraliza el conjunto de decisiones de diseño técnico y estructural que sirven como fundamentos del proyecto. En primer lugar, exponemos los marcos metodológicos utilizados para el manejo del propio ciclo de vida del software, definiendo las herramientas, prácticas y técnicas de ingeniería aplicadas. Luego, seguimos con el desglose de la arquitectura del sistema, delimitando expresamente la separación de responsabilidades a través de las distintas capas de la aplicación: la capa de control y enrutamiento, la capa de persistencia y acceso a datos, y finalmente, el mecanismo de integración con la interfaz de usuario. El principal objetivo de esta arquitectura fue garantizar un sistema modular, escalable, mantenible y flexible en la integración de base de datos.

3.1. Metodología de Ingeniería del Software

Este Trabajo de Fin de Grado se ha organizado por principios de Ingeniería de Software fundamentados en el paradigma ágil. Como se estableció en el plan de trabajo, hemos optado por **Scrum** como referencia, adaptando sus características a las particularidades de un equipo reducido de dos personas. La adaptación resultó esencial para mantener la disciplina y estructura ofrecida por el modelo ágil sin introducir una sobrecarga innecesaria.

Por eso la aplicación de **Scrum** se centró principalmente en ciclos de desarrollo iterativos (*sprints*) de corta duración, donde se planteaban objetivos concretos y alcanzables. En general, toda iteración coincidía con un incremento funcional del sistema, con excepción de las fases iniciales, en lo cual la mayor parte del esfuerzo se mantuvo en una especificación inicial del motor de navegación. Este abordaje permitió ejecutar una validación progresiva de los componentes críticos del proyecto. Específicamente, se ha notado un gran impacto en los *milestones*: motor de navegación en Java, la integración con la base de datos TMDB y la posterior exposición de funcionalidades mediante una API REST.

Desarrollo iterativo e incremental

A cambio de definir completa y decididamente la arquitectura desde del principio con un enfoque lineal y secuencial, como la metodología cascada, el sistema se ha desarrollado de forma incremental. En sus primeras iteraciones, hemos priorizado la validación del propio modelo de navegación y la estructura de datos adyacente, al mismo tiempo que en fases posteriores hemos incorporado progresivamente la capa de acceso a datos, la API y el *frontendweb*. Dicho tratamiento resultó significativamente relevante debido a la naturaleza exploratoria del proyecto, ya que permitió adaptar el diseño del motor de navegación al mismo paso en que se probaba con datos reales.

Gestión de requisitos y backlog

Las funcionalidades más complejas iniciales, como el sistema de filtrado relacional, la gestión de estados de navegación y la combinación de resultados, se desarrollaron mediante periodos de trabajo intensivo centrados en la implementación y validación de cada componente. En cambio, el resto de tareas de menor complejidad o con menor dependencia técnica se fueron abordando progresivamente siguiendo la prioridad establecida en el *backlog*.

Los criterios de priorización y la planificación de las tareas se realizaron en coordinación con la tutoría del proyecto, llevando a cabo revisiones periódicas y ajustes continuos. De esta forma, los objetivos de cada iteración se adaptaban en función de los resultados obtenidos y de los obstáculos técnicos que iban apareciendo durante el desarrollo.

Trabajo colaborativo en un equipo reducido

Hemos adaptado las prácticas de **Scrum** a un entorno de colaboración directa y continua, ya que el proyecto sería desarrollado por un equipo de dos personas. En lugar de llevar a cabo reuniones formales, se optó por mantener una comunicación frecuente entre los integrantes, manteniendo siempre una división clara y acordada de responsabilidades, fundamental para la paralelización del desarrollo sin perder coherencia en el diseño global. Este método de comunicación y organización facilitó una evolución más eficiente y dinámica del software, con una alta capacidad de reacción frente los problemas técnicos.

Control de versiones y gestión del código fuente

En virtud de la necesidad de mantener la integridad del código y facilitar el trabajo colaborativo, hemos utilizado Git como sistema de control de versiones distribuido. El repositorio principal se alojó en GitHub y se empleó una estrategia de ramificación sencilla : una rama principal (**main**) para el código estable y funcional, y ramas de características (*feature branches*) para el desarrollo de nuevos componentes antes de su integración final. Con esa metodología, hemos podido trabajar en paralelo sin conflictos significativos manteniendo siempre una versión funcional

del sistema.

Buenas prácticas de programación y diseño

Durante el desarrollo hemos dado especial atención a la aplicación de buenas prácticas de programación y diseño software. Se siguieron principios de separación de responsabilidades (*Separation of Concerns*) y modularidad, lo está reflejado en la organización del código en distintos paquetes (por ejemplo, `controller`, `model`, `dao`, `filter`, etc.). Dicha estructura tiende a facilitar la legibilidad del código, su mantenimiento y la posibilidad de ampliación futura sin necesidad de realizar modificaciones significativas o profundas en su arquitectura.

3.2. Arquitectura del Sistema

Para permitir la exposición de la lógica del motor de exploración y su respectivo consumo por parte de aplicaciones, hemos diseñado una arquitectura fundamentada en el modelo Cliente-Servidor. Desde el lado del servidor (*backend*), la aplicación favorece el desacoplamiento al basarse en capas bien definidas. Esta sección se dedica a detallar la capa de control, responsable por enrutar las distintas peticiones HTTP, y la capa de filtros, encargada de controlar las políticas de acceso y seguridad.

Diseño de la API y Controladores

Hemos optado por implementar la interfaz de comunicación entre el cliente web y el motor de navegación usando una API de estilo REST, siguiendo la especificación de *Servlets* de Java (`HttpServlet`). Esta elección viene de priorizar un control directo sobre las peticiones y respuestas HTTP frente al uso de frameworks web de más alto nivel (como Spring Boot), que aunque facilitan el desarrollo de aplicaciones web tradicionales, introducen una mayor complejidad estructural y una capa adicional de abstracción que no resulta necesaria para los objetivos de este proyecto.

El uso directo de `HttpServlet` ha sido necesaria para implementar una capa de comunicación más compacta, donde cada controlador se encarga exclusivamente de recibir los parámetros de la petición, delegar la lógica en el motor de navegación y generar la respuesta correspondiente en formato JSON. Emplear esa estrategia fue importante para mantener una separación explícita entre la lógica de navegación y la capa de acceso web. Además permitió un control más preciso de la gestión de sesiones, el formato de las respuestas y el flujo de las peticiones HTTP.

Para ejecutar la aplicación se ha utilizado Apache Tomcat como contenedor de *Servlets*. Nos hemos inclinado por Tomcat frente a otras opciones principalmente por la naturaleza del proyecto. Tomcat es un contenedor considerablemente leve, enfocado en la ejecución de aplicaciones basadas en *Servlets* y JSP, lo que lo convierte en una solución adecuada para trabajar directamente con la API de Java sin generar dependencias adicionales.

Por motivos semejantes, hemos descartado servidores de aplicaciones completos como GlassFish, ya que requiere tecnologías adicionales como EJB y gestión

avanzada de recursos. Alternativas como Jetty también permiten ejecutar *Servlets*, pero está más orientado a una integración embebida dentro de otras aplicaciones/frameworks. Un contenedor independiente, sencillo de configurar y con respaldo de experiencia en entornos académicos y de desarrollo hace que Tomcat resulte la opción más estable encontrada.

Como contrapartida, esta decisión implica un mayor esfuerzo de implementación manual, especialmente en tareas como el enrutamiento de peticiones, la serialización de datos o la gestión de errores. Sin embargo, este *trade-off* se ha asumido de forma consciente, ya que el objetivo principal del proyecto no es el desarrollo de una aplicación web en sí, sino el diseño e implementación de un motor de navegación sobre colecciones de datos, siendo la API web únicamente un mecanismo para exponer dicha funcionalidad.

Gestión del Estado de Navegación (Enfoque *Stateful*)

Componentes transversales de la arquitectura

3.3. Persistencia y Acceso a Datos

3.4. Integración API

Capítulo 4

Desarrollo e Implementación

- 4.1. Representación del contexto de exploración
- 4.2. Filtrado clásico y relacional
- 4.3. Navegación transversal
- 4.4. Operaciones de conjuntos
- 4.5. Obstáculos encontrados y soluciones implementadas

Capítulo 5

Pruebas y Validación

5.1. Casos de uso validados

Capítulo 6

Conclusiones y Trabajo Futuro

Conclusiones del trabajo y líneas de trabajo futuro.

Antes de la entrega de actas de cada convocatoria, en el plazo que se indica en el calendario de los trabajos de fin de grado, el estudiante entregará en el Campus Virtual la versión final de la memoria en PDF.

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and Future Work

Conclusions and future lines of work. This chapter contains the translation of Chapter 6.

Contribuciones Personales

En caso de trabajos no unipersonales, cada participante indicará en la memoria su contribución al proyecto con una extensión de al menos dos páginas por cada uno de los participantes.

En caso de trabajo unipersonal, elimina esta página en el fichero `TFGTeXiS.tex` (comenta o borra la línea `\include{Capitulos/ContribucionesPersonales}`).

Estudiante 1

Al menos dos páginas con las contribuciones del estudiante 1.

Estudiante 2

Al menos dos páginas con las contribuciones del estudiante 2. En caso de que haya más estudiantes, copia y pega una de estas secciones.

Bibliografía

*Y así, del mucho leer y del poco dormir, se
le secó el cerebro de manera que vino a
perder el juicio.*
(modificar en Cascaras\bibliografia.tex)

Miguel de Cervantes Saavedra

BAUTISTA, T., OETIKER, T., PARTL, H., HYNA, I. y SCHLEGL, E. *Una Descripción de $\text{\LaTeX}$ 2 ϵ* . Versión electrónica, 1998.

KNUTH, D. E. *The $\text{\TeX}$ book*. Addison-Wesley Professional., 1986.

KRISHNAN, E., editor. *$\text{\LaTeX}$ Tutorials. A primer*. Indian $\text{\TeX}$ Users Group, 2003.

LAMPORT, L. *$\text{\LaTeX}$: A Document Preparation System, 2nd Edition*. Addison-Wesley Professional, 1994.

MITTELBACH, F., GOOSSENS, M., BRAAMS, J., CARLISLE, D. y ROWLEY, C. *The $\text{\LaTeX}$ Companion*. Addison-Wesley Professional, segunda edición, 2004.

OETIKER, T., PARTL, H., HYNA, I. y SCHLEGL, E. *The Not So Short Introduction to $\text{\LaTeX}$ 2 ϵ* . Versión electrónica, 1996.

Apéndice A

Título del Apéndice A

Los apéndices son secciones al final del documento en las que se agrega texto con el objetivo de ampliar los contenidos del documento principal.

Apéndice	B
----------	----------

Título del Apéndice B

Se pueden añadir los apéndices que se consideren oportunos.

Este texto se puede encontrar en el fichero Cascaras/fin.tex. Si deseas eliminarlo, basta con comentar la línea correspondiente al final del fichero TFGTeXiS.tex.

*–¿Qué te parece desto, Sancho? – Dijo Don Quijote –
Bien podrán los encantadores quitarme la ventura,
pero el esfuerzo y el ánimo, será imposible.*

*Segunda parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

*–Buena está – dijo Sancho –; fírmela vuestra merced.
–No es menester firmarla – dijo Don Quijote–,
sino solamente poner mi rúbrica.*

*Primera parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

